

Unit I

What is Traditional Computing?

Traditional Computing, as name suggests, is a possess of using physical data centers for storing digital assets and running complete networking system for daily operations. In this, access to data, or software, or storage by users is limited to device or official network they are connected with. In this computing, user can have access to data only on system in which data is stored.

Advantages of Traditional Computing

Following are the advantages of Traditional Computing:

- **Control:** With traditional computing, an organization has full control over the hardware and software it uses, allowing for customization and optimization of the computing environment.
- **Security:** Traditional computing offers a high level of data security, as sensitive data can be stored on-premises and protected by firewalls, encryption, and other security measures.
- **Reliability:** Traditional computing is not dependent on internet connectivity, making it less vulnerable to service disruptions or data loss.
- **Compatibility:** Traditional computing environments can be tailored to meet the specific needs of an organization, ensuring compatibility with existing software and systems.
- **Data Ownership:** With traditional computing, an organization owns and controls all of its data, reducing concerns about data privacy and regulatory compliance.

Disadvantages of Traditional Computing

Following are the disadvantages of Traditional Computing:

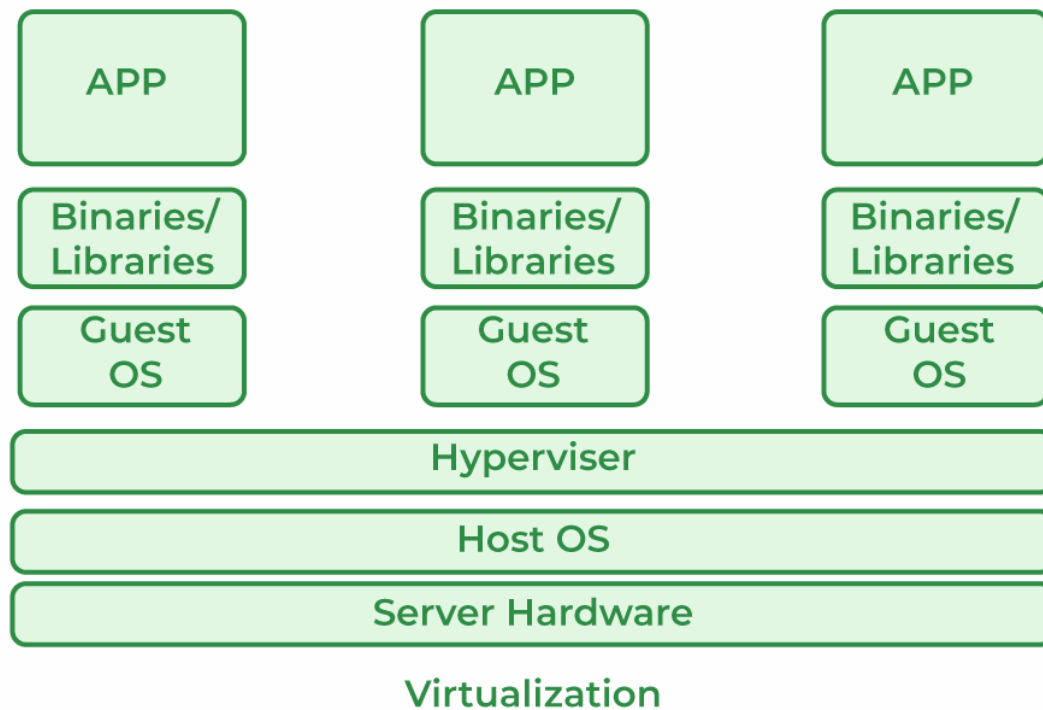
- **Cost:** Traditional computing can be more expensive than cloud computing, as it requires significant capital expenditures for hardware and software, as well as ongoing maintenance and support expenses.
- **Scalability:** Traditional computing can be difficult to scale up or down to meet changing needs, as it requires additional hardware or software to be added to the environment.
- **Accessibility:** Traditional computing may not allow for remote access to applications and data, limiting the ability of users to work from anywhere.
- **Maintenance:** Traditional computing environments require ongoing maintenance and upgrades to ensure security and performance, which can be time-consuming and expensive.
- **Limited Storage Capacity:** Traditional computing environments may have limited storage capacity, requiring organizations to periodically purchase additional hardware to accommodate growing data volumes.

Difference between Cloud Computing and Traditional Computing

Aspect	Cloud Computing	Traditional Computing
Definition	Cloud Computing refers to delivery of different services such as data and programs through internet on different servers.	Traditional Computing refers to delivery of different services on local server.
Infrastructure Location	Cloud Computing takes place on third-party servers that is hosted by third-party hosting companies.	Traditional Computing takes place on physical hard drives and website servers.
Data Accessibility	Cloud Computing is ability to access data anywhere at any time by user.	User can access data only on system in which data is stored.
Cost Effectiveness	Cloud Computing is more cost effective as compared to tradition computing as operation and maintenance of server is shared among several parties that in turn reduce cost of public services.	Traditional Computing is less cost effective as compared to cloud computing because one has to buy expensive equipment's to operate and maintain server.
User-Friendliness	Cloud Computing is more user-friendly as compared to traditional computing because user can have access to data anytime anywhere using internet.	Traditional Computing is less user-friendly as compared to cloud computing because data cannot be accessed anywhere and if user has to access data in another system, then he need to save it in external storage medium.
Internet Dependency	Cloud Computing requires fast, reliable and stable internet connection to access information anywhere at any time.	Traditional Computing does not require any internet connection to access data or information.

Storage and Computing Power	Cloud Computing provides more storage space and servers as well as more computing power so that applications and software run must faster and effectively.	Traditional Computing provides less storage as compared to cloud computing.
Scalability and Elasticity	Cloud Computing also provides scalability and elasticity i.e., one can increase or decrease storage capacity, server resources, etc., according to business needs.	Traditional Computing does not provide any scalability and elasticity.
Maintenance and Support	Cloud service is served by provider's support team.	Traditional Computing requires own team to maintain and monitor system that will need a lot of time and efforts.
Software Delivery Model	Software is offered as an on-demand service (SaaS) that can be accessed through subscription service.	Software is purchased individually for every user and requires to be updated periodically.

Virtualization in Cloud Computing and Types



Virtualization is the process of creating a virtual representation of hardware such as server, storage, network or other physical machines. It supports multiple copies of virtual machines (VMs) to execute on one physical machine each with their own operating system and programs. This optimizes hardware efficiency and flexibility and enables resources to be shared between multiple customers or organizations.

Virtualization is a key to providing Infrastructure as a Service (IaaS) solutions for cloud computing, whereby the user has access to remote computing resources.

Example

Think of a big house where only one person lives. Most of the rooms are empty and not being used. Now, if you are dividing the house into smaller rooms and renting each one to different people. Everyone gets their own space to live, cook, and sleep all inside the same house.

Let's try to understand this example how this house behaves like virtualization:

- The **house** is like one real computer.
- The **rooms** are like virtual machines.
- The **people** are like different users or apps.

Instead of buying a new house for each person, you are using one house in a smarter way just like virtualization lets one computer do the work of many.

Let us understand virtualization by taking a real-world example:

Suppose there is a company that requires servers for four different purposes:

- Store customer data securely

- Host an online shopping website
- Process employee payroll systems
- Run Social media campaign software for marketing

All these tasks require different things:

- The customer data server requires a lot of space and a Windows operating system.
- The online shopping website requires a high-traffic server and needs a Linux operating system.
- The payroll system requires greater internal memory (RAM) and must use a certain version of the operating system.
- The marketing software demands specialized software tools and needs plenty of processing power.

In order to fulfill these requirements, the company initially configures **four individual physical servers**, each for a different purpose. This implies that the company needs to purchase four servers, keep them running, and upgrade them individually, which is very expensive. The company also cannot utilize the full capacity of each server, so it's paying for more than it requires, wasting some of the servers' capabilities.

Now, by utilizing **virtualization**, the company can run these four applications on a few physical servers through multiple virtual machines (VMs). Each VM will behave as an independent server, possessing its own operating system and resources. Through this means, the company can cut down on expenses, conserve resources, and manage everything from a single location with ease.

Need of Virtualization.

Virtualization is important because it lets you get the most out of your computer or server resources. Consider it like being able to use one physical box as many smaller, independent "virtual" boxes. There are multiple virtual boxes, each having its own program to run and data to store, but they use the same physical box.

1. Better use of Resources

Instead of allowing for numerous unused machines, virtualization enables you to host multiple programs or systems on one computer, which is more effective.

2. Cost Utilization

Companies can save their money on hardware, power, and maintenance by using less physical equipment.

3. Flexibility

Virtual machines can be easily installed, relocated and resized to suit changing requirements. If a virtual machine requires more power, it can obtain it rapidly without requiring new hardware.

4. Security

Virtualization isolates various applications or systems from each other, so if one of them has an issue, it won't affect others.

5. Simple Recovery

In case something goes wrong, it's simple to back up or restore virtual machines, allowing companies to return to work quickly after an issue.

How does Virtualization Work

Virtualizations uses special software known as hypervisor, to create many virtual computers (cloud instances) on one physical computer. The Virtual Machines behave like actual computers but use the same physical machine.

Virtual Machines (Cloud Instances)

After installing virtualization software on your computer, you can set up one or more virtual machines. The Virtual machines are similar to other applications on your computer. The Actual computer is "**Host**" and the Virtual Computers are "**Guests**". You can have several guests on a single host, and each guest can have its own operating system, which may be the same or different from the host.

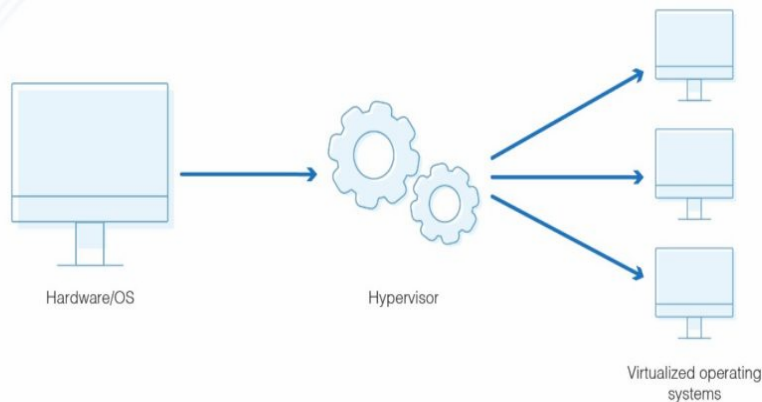
As a user you can see that every virtual machine behaves as a normal computer. It has its own configurations, programs, and settings. The resources, such as the processor (CPU), Memory (RAM), and storage, are all accessed by the virtual machines, but they appear and function exactly like they would in a real computer. You can update or modify the virtual machine's operating system and applications without touching the original computer.

Hypervisors

A hypervisor is the software that gets virtualization to work. It serves as an intermediary between the physical computer and the virtual machines. The hypervisor controls the virtual machines' use of the physical resources (such as the CPU and memory) of the host computer.

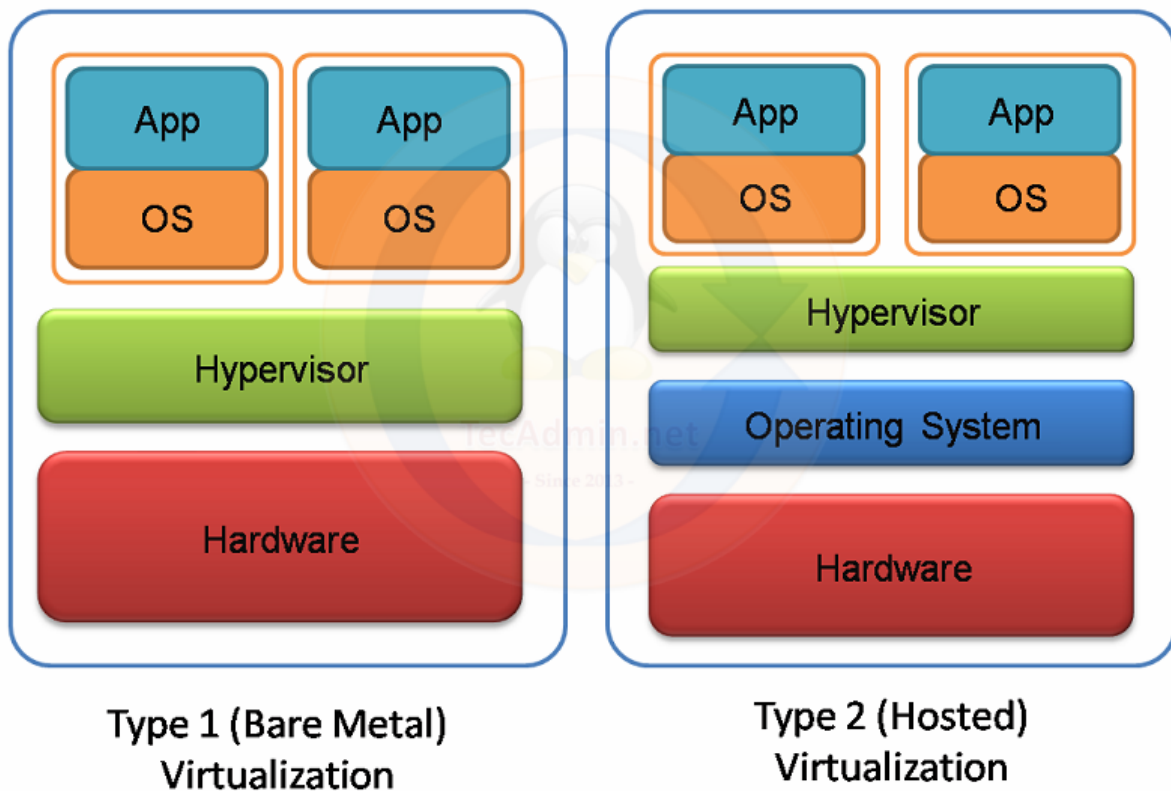
For instance, if one virtual machine wants additional computing capability, it requests it from the hypervisor. The hypervisor ensures the request is forwarded to the physical hardware, and it's accomplished.

What Is a Hypervisor?



There exist two categories of hypervisors:

Type-1 Hypervisor vs Type-2 Hypervisor:



Type — 1 vs Type 2 Virtualization

Type-1 Hypervisor (Bare Metal Hypervisor):

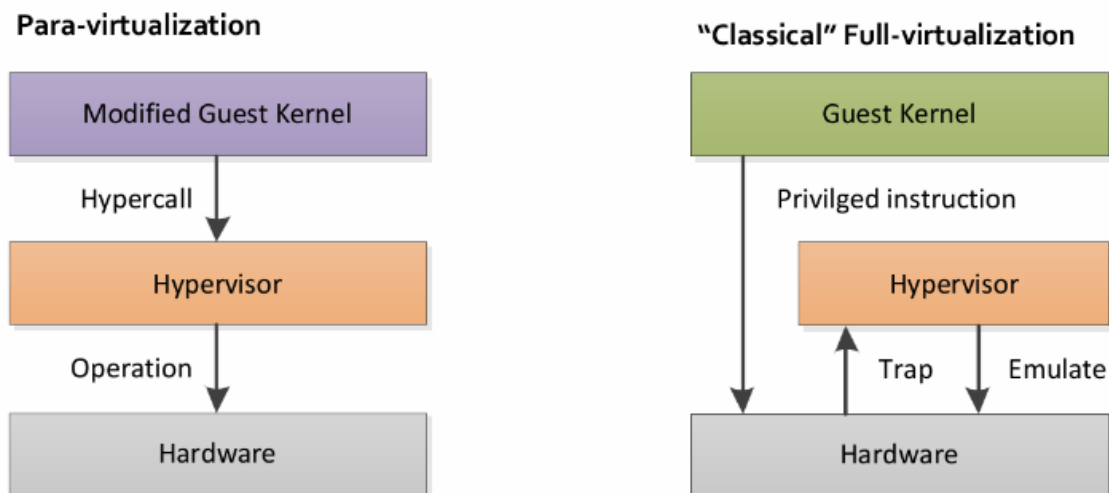
- Type-1 hypervisors run directly on the physical hardware without the need for a host operating system.

- They have direct access to the underlying hardware resources, offering better performance and efficiency compared to Type-2 hypervisors.
- Type-1 hypervisors are commonly used in enterprise data centers and cloud environments.
- Examples include VMware vSphere/ESXi, Microsoft Hyper-V, and KVM (Kernel-based Virtual Machine).

Type-2 Hypervisor (Hosted Hypervisor):

- Type-2 hypervisors run on top of a host operating system, which in turn runs on the physical hardware.
- They rely on the host operating system for managing hardware resources and providing device drivers.
- Type-2 hypervisors are typically used for desktop or development environments.
- Examples include Oracle VirtualBox, VMware Workstation, and Parallels Desktop.

Full Virtualization vs Para Virtualization:



Para vs Full Virtualization

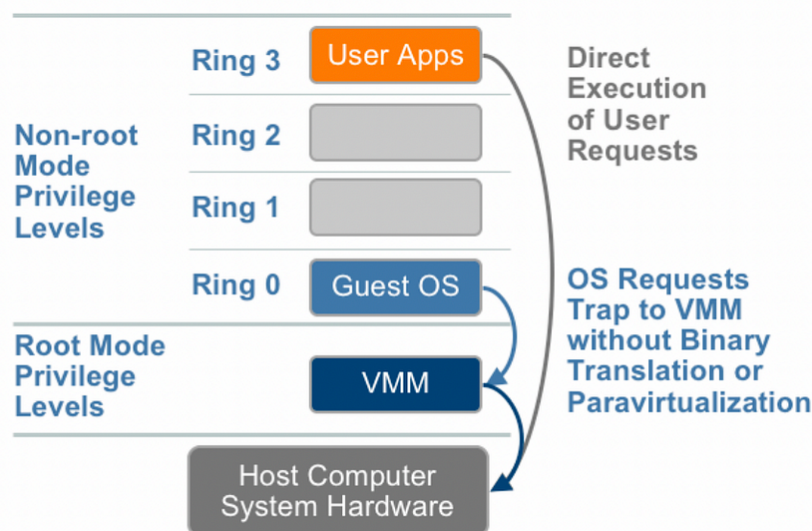
Full Virtualization:

- In full virtualization, the guest operating system is unaware that it's running in a virtualized environment.
- The hypervisor presents a virtualized hardware environment to the guest operating system, mimicking the underlying physical hardware.
- Full virtualization provides broad compatibility with unmodified guest operating systems but may incur performance overhead due to hardware emulation.
- Examples of full virtualization include VMware ESXi with Intel VT-x/AMD-V hardware-assisted virtualization.

Para Virtualization:

- In para virtualization, the guest operating system is aware of its virtualized environment and interacts with the hypervisor through special paravirtualized drivers.
- Paravirtualization eliminates the need for hardware emulation, resulting in lower overhead and improved performance compared to full virtualization.
- However, para virtualization requires modifications to the guest operating system to support paravirtualized drivers.
- Examples of para virtualization include Xen hypervisor with Xen-aware guest operating systems.

Hardware Assisted Virtualization:

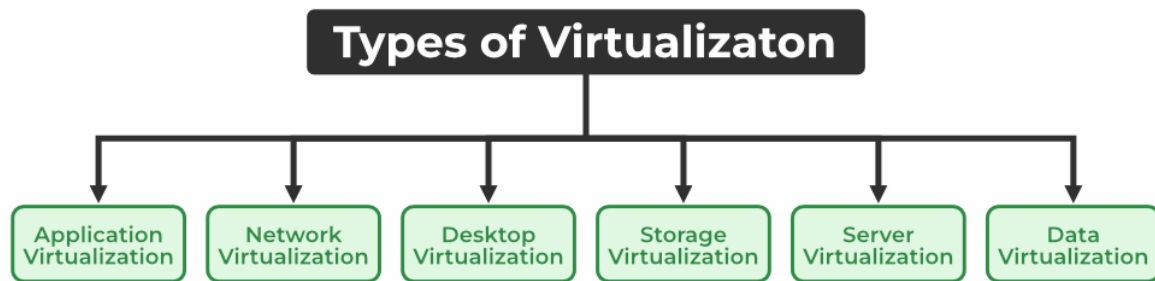


- Hardware vendors are rapidly embracing virtualization and developing new features to simplify virtualization techniques.
- First generation enhancements include Intel Virtualization Technology (VT-x) and AMD's AMD-V which both target privileged instructions with a new CPU execution mode feature that allows the VMM to run in a new root mode below ring 0.
- Privileged and sensitive calls are set to automatically trap to the hypervisor, removing the need for either binary translation or paravirtualization.
- The guest state is stored in Virtual Machine Control Structures (VT-x) or Virtual Machine Control Blocks (AMD-V).

Types of Virtualizations

1. Application Virtualization
1. Network Virtualization
1. Desktop Virtualization
1. Storage Virtualization
1. Server Virtualization

1. Data virtualization



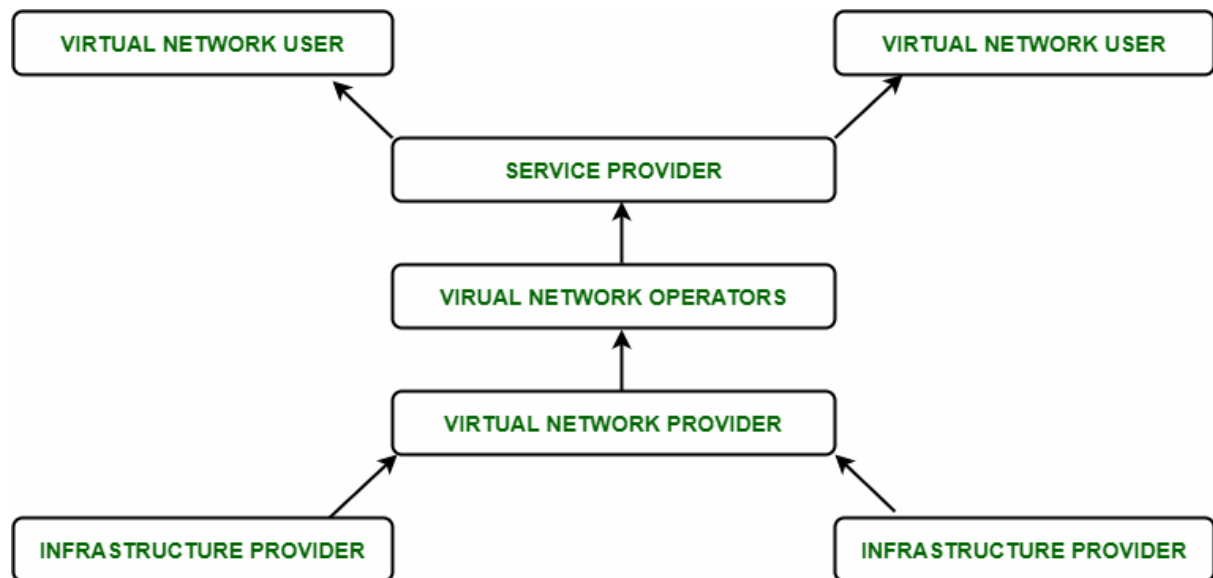
Types of Virtualizations

1. Application Virtualization: Application virtualization enables remote access by which users can directly interact with deployed applications without installing them on their local machine. Your personal data and the applications settings are stored on the server, but you can still run it locally via the internet. It's useful if you need to work with multiple versions of the same software. Common examples include hosted or packaged apps.

Example: Microsoft Azure lets people use their applications without putting them on their own computers. Once this application is setup in the cloud then employees can use it from any device, like a laptop or tablet. It feels like the application is on their computer, but it's really running on Azure's servers. This makes things easier, faster, and safer for the company.

2. Network Virtualization: This allows multiple virtual networks to run on the same physical network, each operating independently. You can quickly set up virtual switches, routers, firewalls, and VPNs, making network management more flexible and efficient.

Example: Google Cloud is an example of Network Virtualization. Companies create their own networks using software instead of physical devices with the help of Google Cloud. They can set up things like IP addresses, firewalls, and private connections all in the cloud. This makes it easy to manage, change, and grow their network without buying any hardware. It saves time, money, and gives more flexibility.



Network Virtualization

3. Desktop Virtualization: Desktop virtualization is a process in which you can create different virtual desktops that users can use from any device like laptop, tablet. It's great for users who need flexibility, as it simplifies software updates and provides portability.

Example: GeeksforGeeks is a Edtech company which uses services like **Amazon WorkSpaces** or **Google Cloud (GCP) Virtual Desktops** to give its team members access to the same coding setup with all the tools they required for the easy access of this team work. Now their team members can easily log in from any device like a laptop, tablet, or even a phone and use a virtual desktop that will run perfectly in the cloud. This makes it easy for GeeksforGeeks company to manage, update, and keep everything secure without requirement of physical computers for everyone.

4. Storage Virtualization: This combines storage from different servers into a single system, making it easier to manage. It ensures smooth performance and efficient operations even when the underlying hardware changes or fails.

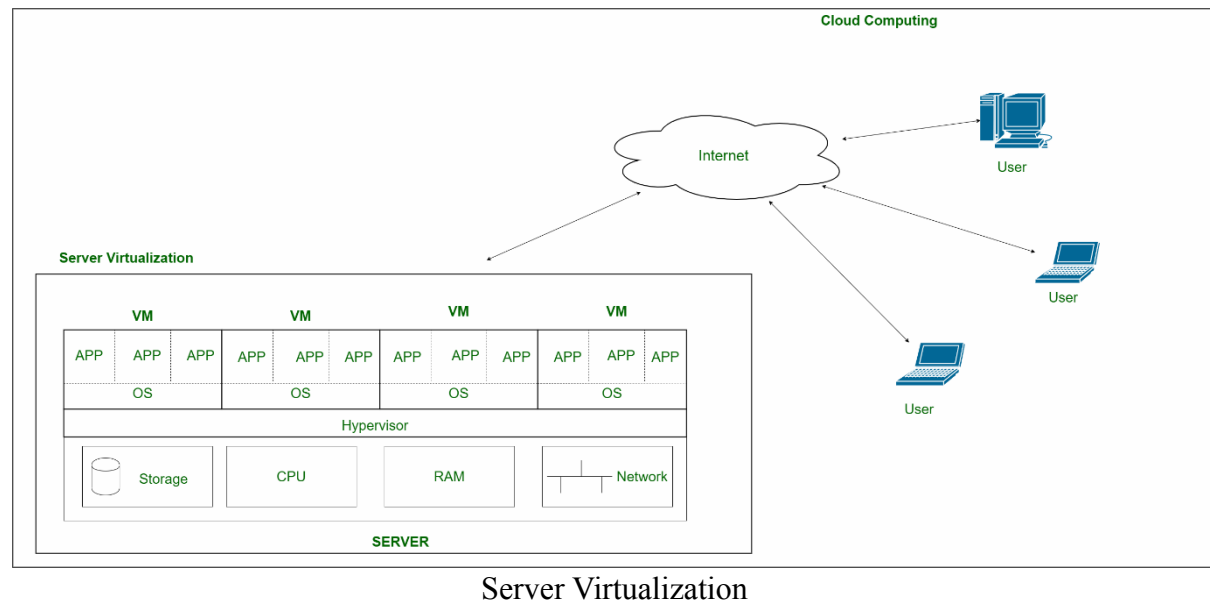
Example: Amazon S3 is an example of storage virtualization because in S3 we can easily store any amount of data from anywhere. Suppose a MNC have lots of files and data of company to store. By Amazon S3 company can store all their files and data in one place and access these from anywhere without any kind of issue in secure way.

5. Server Virtualization: This splits a physical server into multiple virtual servers, each functioning independently. It helps improve performance, cut costs and makes tasks like server migration and energy management easier.

Example: A startup company has a powerful physical server. This company can use server virtualization software like VMware vSphere, Microsoft Hyper-V or KVM to create more virtual machines (VMs) on that one server.

Each VM here is an isolated server, that runs on their own operating system(like Windows and Linux) and run it's own applications. For example, a company might run A web server on one VM, A database server on another VM, A file server on a third VM all on the same

physical machine. This reduces costs, makes it easier to manage and back up servers, and allows quick recovery if one VM fails.



6. Data Virtualization: This brings data from different sources together in one place without needing to know where or how it's stored. It creates a unified view of the data, which can be accessed remotely via cloud services.

Example: Companies like Oracle and IBM offer solutions for this.

Benefits of Virtualization

Here are some of the benefits of using Virtualization in Cloud Computing -

- More flexible and efficient allocation of resources.
- Enhance development productivity.
- It lowers the cost of IT infrastructure.
- Remote access and rapid scalability.
- High availability and disaster recovery.
- Pay per use of the IT infrastructure on demand.
- Enables running multiple operating systems.

Drawback of Virtualization

Virtualization makes our work easy but there are some drawbacks of using virtualization which are follows:

- **High Initial Investment:** Clouds have a very high initial investment, but it is also true that it will help in reducing the cost of companies.
- **Learning New Infrastructure:** As the companies shifted from Servers to Cloud, it requires highly skilled staff who have skills to work with the cloud easily and for this, you have to hire new staff or provide training to current staff.

- **Risk of Data:** Hosting data on third-party resources can lead to putting the data at risk, it has the chance of getting attacked by any hacker or cracker very easily.

Characteristics of Virtualization

The following are characteristics of virtualization:

- **Increased Security:** The ability to control the execution of a guest program in a completely transparent manner opens new possibilities for delivering a secure, controlled execution environment. All the operations of the guest programs are generally performed against the virtual machine, which then translates and applies them to the host programs.
- **Managed Execution:** In particular, sharing, aggregation, emulation, and isolation are the most relevant features.
- **Sharing:** Virtualization allows the creation of a separate computing environment within the same host.
- **Aggregation:** It is possible to share physical resources among several guests, but virtualization also allows aggregation, which is the opposite process.

How is Virtualization Different from Cloud Computing

Below is the table that shows the comparison between virtualization and Cloud Computing:

Aspect	Virtualization	Cloud Computing
What it is	Creating multiple virtual versions of a physical resource	Accessing computing resources (like storage, servers) via the internet
Where it runs	On physical machines or servers	On remote servers provided by cloud service providers
Focus	Efficient use of physical resources	On-demand access to resources over the internet
Example	Running multiple virtual machines on a single physical server	Storing data on Google Drive or renting a virtual machine on AWS
Control	You have more control over the virtual environment	Cloud providers manage the physical hardware; you manage only what you use

Virtualization vs Containerization

Below is a comparison table between Virtualization and Containerization which shows how server virtualization is different from containerization:

Aspect	Virtualization	Containerization
Architecture	Full OS per VM	Shared host OS
Resource Usage	High	Low
Performance	Slightly slower due to overhead	Faster due to lightweight design
Scalability	Limited by heavier VMs	Highly scalable
Portability	Moderate	Very high
Use Case	Running multiple OS types	Rapid deployment of apps

♦ 1. Startup Hosting a Web Application

Scenario: A startup develops an e-commerce website and wants to launch it quickly without investing in physical servers.

Public Cloud Solution:

They host the website on Amazon Web Services (AWS) or Microsoft Azure, using services like:

EC2 for compute

S3 for storage

RDS for database

This gives them flexibility, scalability, and low upfront cost.

♦ 2. Educational Institution for Online Learning

Scenario: A university launches an online learning platform with video lectures, assignments, and live quizzes.

Public Cloud Solution:

They use Google Cloud Platform (GCP) or AWS to host:

Video content (Cloud Storage / S3)

Scalable servers for quizzes (App Engine / Lambda)

Content delivery (Cloud CDN)

♦ 3. Mobile App with Global Users

Scenario: A company builds a mobile social media app with global users.

Public Cloud Solution:

They use:

Firebase (by Google) for real-time database and authentication

AWS CloudFront to deliver images and videos quickly

Azure Cosmos DB for geo-replicated data access

♦ 4. Data Backup and Disaster Recovery

Scenario: A small business wants to back up important customer and financial data securely.

Public Cloud Solution:

They use:

AWS Glacier for low-cost archival

Azure Backup to regularly back up data from on-premise servers

♦ 5. High-Performance Computing (HPC)

Scenario: A research lab needs to run complex simulations that require high processing power.

Public Cloud Solution:

They use:

AWS EC2 High-Performance Instances

Google Cloud TPU (Tensor Processing Unit) for machine learning tasks

They pay only for the time used — no need to buy expensive servers.

♦ 6. Streaming Platform

Scenario: A media company launches an OTT platform like Netflix or Hotstar.

Public Cloud Solution:

They use:

AWS Media Services or Azure Media Services for video processing

CDN to stream videos worldwide efficiently

Auto-scaling during peak traffic times

♦ 7. SaaS Product Deployment

Scenario: A software company launches a Software-as-a-Service (SaaS) product for invoice generation.

Public Cloud Solution:

They use:

Multi-tenant architecture on AWS or Azure

Managed databases, authentication, email services, etc.

♦ 8. DevOps and CI/CD Pipelines

Scenario: A development team needs to continuously build, test, and deploy software.

Public Cloud Solution:

Use GitHub Actions + AWS CodePipeline or Azure DevOps

Automatically deploy to cloud containers (ECS, AKS, etc.)

Here are real-world scenarios based on private cloud usage, where organizations build and use their own cloud infrastructure, often for security, control, and compliance reasons:

♦ 1. Government Agency Data Center

Scenario: A national government agency handles sensitive citizen data (like Aadhaar in India) and needs complete control and security.

Private Cloud Solution:

Builds its own private cloud using OpenStack or VMware vSphere

Hosts all data within government-owned data centers

Complies with data sovereignty and security laws

♦ 2. Healthcare Organization

Scenario: A hospital chain manages patient records, lab reports, and medical imaging, all of which must comply with privacy regulations (like HIPAA).

Private Cloud Solution:

Hosts Electronic Health Records (EHR) on a private cloud

Uses virtualization (e.g., KVM or VMware) to run secure apps

Provides internal secure access for doctors and staff only

♦ 3. Banking & Financial Institutions

Scenario: A bank handles transactions, customer data, and real-time processing — needing high availability and strict security.

Private Cloud Solution:

Builds a secure private cloud to host core banking systems

Uses VMware, OpenStack, or IBM Cloud Private

Ensures compliance with RBI and other financial regulations

♦ 4. Large Enterprise with Global Branches

Scenario: A multinational corporation needs to run HR, finance, and collaboration tools securely across all branches.

Private Cloud Solution:

Deploys private cloud in their data centers

Integrates it with Identity Management (SSO)

Enables internal collaboration without external exposure

♦ 5. Research Lab with Confidential Projects

Scenario: A defense R&D center runs simulations and experiments that must remain classified.

Private Cloud Solution:

Sets up a high-security private cloud environment

Limits network access to only authorized internal users

Uses on-premise computing clusters with virtualization

♦ 6. University with Custom Learning Tools

Scenario: A university wants full control over its e-learning platform, student data, and research outputs.

Private Cloud Solution:

Builds private cloud using Proxmox or OpenStack

Hosts LMS (Learning Management System), research portals, and storage

Avoids third-party data dependency

♦ 7. Telecom Company with Internal Services

Scenario: A telecom provider needs to manage its internal CRM, billing systems, and subscriber data.

Private Cloud Solution:

Uses a private cloud for real-time billing and customer support tools

Ensures service uptime and compliance with telecom regulations

♦ 8. Manufacturing Company with IoT Devices

Scenario: A factory uses IoT-enabled machines and doesn't want production data to go outside.

Private Cloud Solution:

Hosts IoT management platform on private cloud within the factory network

Ensures low-latency and no data leakage to public internet

Test Cases based on Deployment Model scenarios (Identify)

♦ 1. Hospital with Patient Data & Online Services

Scenario: A hospital stores sensitive patient data but also offers online appointment booking and video consultations.

Stores patient health records (EHR), lab reports (compliant with HIPAA)

Hosts website, mobile app, and video consultation services using AWS or Azure
This ensures security and public accessibility.

♦ 2. Bank with On-Prem Core Systems & Scalable Services

Scenario: A bank wants to protect customer data but needs high scalability during online banking traffic surges.

Core banking systems, customer data, internal operations

Mobile banking interface, fraud analytics, or chatbots during high traffic
Allows secure base and elastic scalability.

♦ 3. University with Research and Public Access

Scenario: A university runs internal systems (like student management) but also shares open courses.

Stores student records, exam results, and internal communication

Hosts MOOCs (Massive Open Online Courses) on platforms like Azure or Google Cloud
Keeps internal systems secure, while public content is scalable.

♦ 4. E-Commerce Company during Festival Sales

Scenario: A shopping site handles normal traffic internally but needs extra resources during Diwali or Black Friday sales.

Normal operations, payment gateway integration

Auto-scale web servers during traffic spikes via AWS/Azure
Uses cloud bursting technique to handle peak loads.

♦ 5. Software Company – SaaS + DevOps

Scenario: A software firm builds applications and runs CI/CD pipelines for continuous deployment.

Stores source code, sensitive IP, internal testing

Executes CI/CD pipelines and hosts customer-facing dashboards

Combines control with agility.

♦ **6. Manufacturing Company with Smart Factory**

Scenario: A smart factory collects IoT sensor data and needs secure local processing but also AI analytics.

Local edge computing, machine control systems

Sends selected data to Google Cloud or AWS for analytics or ML model training
Enables faster decisions and long-term analytics.

♦ **7. Government Department – Citizen Services**

Scenario: A government body offers digital services to citizens but retains sensitive national data securely.

Hosts national identity, legal, or security-related records

Hosts public service portals, complaint redressal, mobile apps
Ensures governance and citizen accessibility.

♦ **8. Media Company – Internal Editing & Global Streaming**

Scenario: A media company edits content locally but streams it globally.

Video editing, raw file storage, pre-release content

Uses CDN (AWS CloudFront or Azure CDN) to stream content
Combines confidentiality with global delivery.

Test Cases based on Deployment Model scenarios (Identify)

1. Group of Universities Sharing Research Resources

Scenario: Several universities collaborate on scientific research and need a common platform to store data, simulations, and findings.

They share a cloud hosted by a government or education agency

Common access to research data, e-libraries, HPC clusters

Maintains data privacy while supporting academic collaboration

♦ 2. Government Departments Sharing Infrastructure

Scenario: Different government departments (like health, transport, and agriculture) need to collaborate on a digital governance platform.

A government agency sets up a cloud for shared applications (e.g., public grievance systems, census data)

Shared authentication, security, and compliance policies

Examples: India's MeghRaj Cloud, UK's G-Cloud

♦ 3. Healthcare Providers Sharing Patient Data

Scenario: A group of hospitals and clinics in a region collaborate to manage patient referrals, diagnostics, and treatments.

Shared infrastructure managed by a healthcare authority

Common EHR (Electronic Health Records) platform

Compliant with privacy laws (like HIPAA or NDHM in India)

♦ 4. Banking Consortium for Regulatory Compliance

Scenario: A group of banks create a joint KYC (Know Your Customer) and anti-fraud monitoring platform.

Shared data analytics, compliance tools, and authentication services

Ensures cost-effectiveness and common regulatory standards (e.g., RBI, SEBI)

Facilitates fraud detection across institutions

♦ 5. Law Enforcement Agencies Across States

Scenario: Police departments from multiple states share a cloud platform for tracking criminals, FIRs, and intelligence.

Managed by central or state government

Secure, auditable, and access-controlled

Enables cooperation while protecting sensitive data

♦ **6. NGO Network for Social Services**

Scenario: A group of NGOs working on child welfare, education, and women's rights share case management systems.

Jointly hosted cloud platform with user roles and permissions

Shared dashboards, progress tracking, and funding reports

Cost-effective and secure collaboration

♦ **7. Smart City Collaboration**

Scenario: Multiple municipal corporations share a platform for urban data: traffic, pollution, water management, etc.

Managed by a regional Smart City mission or urban development authority

Supports data-driven policy and infrastructure management

Enables integration of different city systems

♦ **8. Educational Boards Sharing Curriculum Platforms**

Scenario: Central and state education boards need a common platform for exams, syllabi, and student records.

Shared learning management and assessment tools

Secure storage of marksheets and results

Supports multilingual and regional customization

Feature	Container	Virtual machine
Virtualization level	Operating system Level	Hardware level
Operating system	Shares host OS kernel	Guest OS (full OS instance)
Size	Megabytes (MBs)	Gigabytes (GBs)
Boot time	Seconds	Minutes
Resource usage	Lower	Higher
Isolation	Process-level	Hardware-level
Portability	Highly portable	Less portable

Management	Typically managed by container orchestration tools (for example Kubernetes)	Managed by hypervisors (e.g., VMware, KVM) or cloud management platforms
Use cases	Microservices, web applications, CI/CD pipelines, cloud-native applications	Legacy applications, applications requiring strong isolation, diverse OS requirements

Cluster Architecture

The architectural concepts behind Kubernetes.

A Kubernetes cluster consists of a control plane plus a set of worker machines, called nodes, that run containerized applications. Every cluster needs at least one worker node in order to run Pods.

The worker node(s) host the Pods that are the components of the application workload. The control plane manages the worker nodes and the Pods in the cluster. In production environments, the control plane usually runs across multiple computers and a cluster usually runs multiple nodes, providing fault-tolerance and high availability.

This document outlines the various components you need to have for a complete and working Kubernetes cluster.

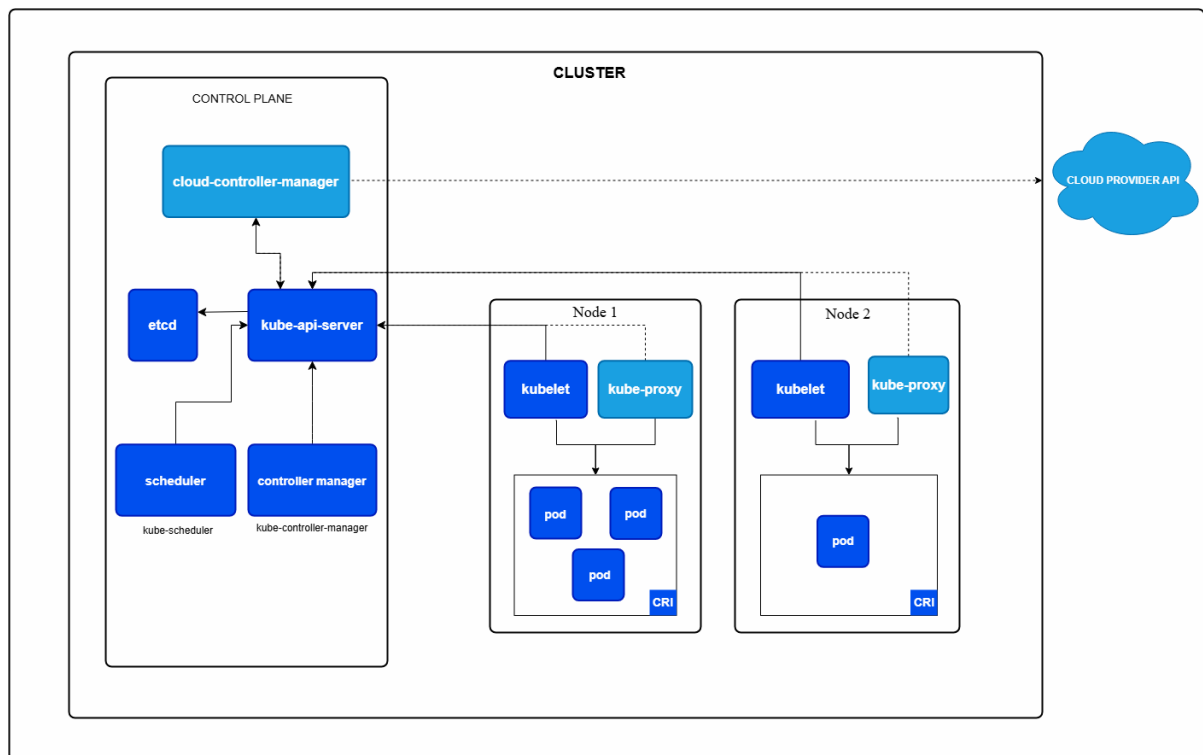


Figure 1. Kubernetes cluster components.

About this architecture

Control plane components

The control plane's components make global decisions about the cluster (for example, scheduling), as well as detecting and responding to cluster events (for example, starting up a new [pod](#) when a Deployment's [replicas](#) field is unsatisfied).

Control plane components can be run on any machine in the cluster. However, for simplicity, setup scripts typically start all control plane components on the same machine, and do not run user containers on this machine. See [Creating Highly Available clusters with kubeadm](#) for an example control plane setup that runs across multiple machines.

kube-apiserver

The API server is a component of the Kubernetes [control plane](#) that exposes the Kubernetes API. The API server is the front end for the Kubernetes control plane.

The main implementation of a Kubernetes API server is [kube-apiserver](#). kube-apiserver is designed to scale horizontally—that is, it scales by deploying more instances. You can run several instances of kube-apiserver and balance traffic between those instances.

etcd

Consistent and highly-available key value store used as Kubernetes' backing store for all cluster data.

If your Kubernetes cluster uses etcd as its backing store, make sure you have a [back up](#) plan for the data.

You can find in-depth information about etcd in the official [documentation](#).

kube-scheduler

Control plane component that watches for newly created [Pods](#) with no assigned [node](#), and selects a node for them to run on.

Factors taken into account for scheduling decisions include: individual and collective [resource](#) requirements, hardware/software/policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference, and deadlines.

kube-controller-manager

Control plane component that runs [controller](#) processes.

Logically, each [controller](#) is a separate process, but to reduce complexity, they are all compiled into a single binary and run in a single process.

There are many different types of controllers. Some examples of them are:

- Node controller: Responsible for noticing and responding when nodes go down.
- Job controller: Watches for Job objects that represent one-off tasks, then creates Pods to run those tasks to completion.
- EndpointSlice controller: Populates EndpointSlice objects (to provide a link between Services and Pods).
- ServiceAccount controller: Create default ServiceAccounts for new namespaces.

The above is not an exhaustive list.

cloud-controller-manager

A Kubernetes [control plane](#) component that embeds cloud-specific control logic. The cloud controller manager lets you link your cluster into your cloud provider's API, and separates out the components that interact with that cloud platform from components that only interact with your cluster.

The cloud-controller-manager only runs controllers that are specific to your cloud provider. If you are running Kubernetes on your own premises, or in a learning environment inside your own PC, the cluster does not have a cloud controller manager.

As with the kube-controller-manager, the cloud-controller-manager combines several logically independent control loops into a single binary that you run as a single process. You can scale horizontally (run more than one copy) to improve performance or to help tolerate failures.

The following controllers can have cloud provider dependencies:

- Node controller: For checking the cloud provider to determine if a node has been deleted in the cloud after it stops responding
- Route controller: For setting up routes in the underlying cloud infrastructure
- Service controller: For creating, updating and deleting cloud provider load balancers

Node components

Node components run on every node, maintaining running pods and providing the Kubernetes runtime environment.

kubelet

An agent that runs on each [node](#) in the cluster. It makes sure that [containers](#) are running in a [Pod](#).

The [kubelet](#) takes a set of PodSpecs that are provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy. The kubelet doesn't manage containers which were not created by Kubernetes.

kube-proxy (optional)

kube-proxy is a network proxy that runs on each [node](#) in your cluster, implementing part of the Kubernetes [Service](#) concept.

[kube-proxy](#) maintains network rules on nodes. These network rules allow network communication to your Pods from network sessions inside or outside of your cluster. kube-proxy uses the operating system packet filtering layer if there is one and it's available. Otherwise, kube-proxy forwards the traffic itself.

If you use a [network plugin](#) that implements packet forwarding for Services by itself, and providing equivalent behavior to kube-proxy, then you do not need to run kube-proxy on the nodes in your cluster.

Container runtime

A fundamental component that empowers Kubernetes to run containers effectively. It is responsible for managing the execution and lifecycle of containers within the Kubernetes environment.

Kubernetes supports container runtimes such as [containerd](#), [CRI-O](#), and any other implementation of the [Kubernetes CRI \(Container Runtime Interface\)](#).

Addons

Addons use Kubernetes resources ([DaemonSet](#), [Deployment](#), etc) to implement cluster features. Because these are providing cluster-level features, namespaced resources for addons belong within the kube-system namespace.

Selected addons are described below; for an extended list of available addons, please see [Addons](#).

DNS

While the other addons are not strictly required, all Kubernetes clusters should have [cluster DNS](#), as many examples rely on it.

Cluster DNS is a DNS server, in addition to the other DNS server(s) in your environment, which serves DNS records for Kubernetes services.

Containers started by Kubernetes automatically include this DNS server in their DNS searches.

Web UI (Dashboard)

[Dashboard](#) is a general purpose, web-based UI for Kubernetes clusters. It allows users to manage and troubleshoot applications running in the cluster, as well as the cluster itself.

Container resource monitoring

[Container Resource Monitoring](#) records generic time-series metrics about containers in a central database, and provides a UI for browsing that data.

Cluster-level Logging

A [cluster-level logging](#) mechanism is responsible for saving container logs to a central log store with a search/browsing interface.

Network plugins

[Network plugins](#) are software components that implement the container network interface (CNI) specification. They are responsible for allocating IP addresses to pods and enabling them to communicate with each other within the cluster.

Architecture variations

While the core components of Kubernetes remain consistent, the way they are deployed and managed can vary. Understanding these variations is crucial for designing and maintaining Kubernetes clusters that meet specific operational needs.

Control plane deployment options

The control plane components can be deployed in several ways:

Traditional deployment

Control plane components run directly on dedicated machines or VMs, often managed as systemd services.

Static Pods

Control plane components are deployed as static Pods, managed by the kubelet on specific nodes. This is a common approach used by tools like kubeadm.

Self-hosted

The control plane runs as Pods within the Kubernetes cluster itself, managed by Deployments and StatefulSets or other Kubernetes primitives.

Managed Kubernetes services

Cloud providers often abstract away the control plane, managing its components as part of their service offering.

Workload placement considerations

The placement of workloads, including the control plane components, can vary based on cluster size, performance requirements, and operational policies:

- In smaller or development clusters, control plane components and user workloads might run on the same nodes.
- Larger production clusters often dedicate specific nodes to control plane components, separating them from user workloads.
- Some organizations run critical add-ons or monitoring tools on control plane nodes.

Cluster management tools

Tools like kubeadm, kops, and Kubespray offer different approaches to deploying and managing clusters, each with its own method of component layout and management.

The flexibility of Kubernetes architecture allows organizations to tailor their clusters to specific needs, balancing factors such as operational complexity, performance, and management overhead.

Customization and extensibility

Kubernetes architecture allows for significant customization:

- Custom schedulers can be deployed to work alongside the default Kubernetes scheduler or to replace it entirely.
- API servers can be extended with CustomResourceDefinitions and API Aggregation.
- Cloud providers can integrate deeply with Kubernetes using the cloud-controller-manager.

The flexibility of Kubernetes architecture allows organizations to tailor their clusters to specific needs, balancing factors such as operational complexity, performance, and management overhead.